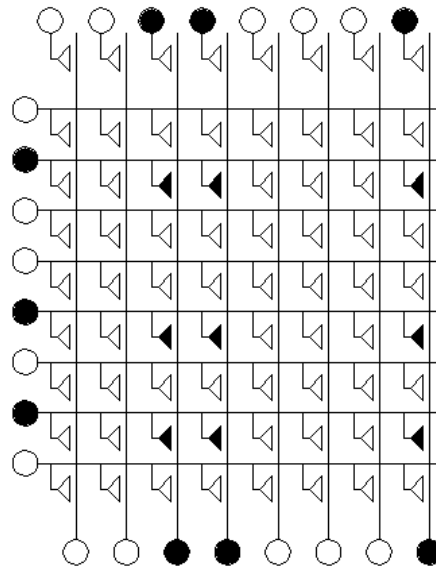


Neural Networks and Machine Learning

Simple associative networks



Associative memory

The task of an associative memory is to learn associations between input and output patterns.

Store a set of input-output pairs $\{(\mathbf{x}^1, \mathbf{y}^1), (\mathbf{x}^2, \mathbf{y}^2), \dots\}$.

Respond with output $\mathbf{y}^p$ to presentations of previously stored input $\mathbf{x}^p$.

Example: telephone directory. Store a set of associations between names and phone numbers:

(Yi Sun, 5063)

(Shabnam Kadir, 3368)

(Volker Steuber, 4350)

When the directory is presented with a name it retrieves the corresponding phone number:

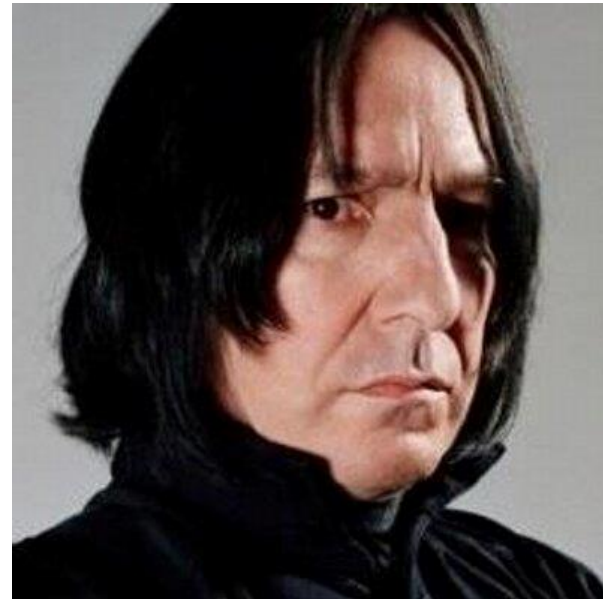
Yi Sun? >> 5063

More examples of associative memory

Other examples:

Store a collection of web pages. Retrieve one of the web pages by googling a text fragment on it.

Remember a number of faces. Recognise a face although only a part of it is visible.



Auto-associative and hetero-associative memory

The three previous examples illustrate two forms of associative memory:

Hetero-associative memory: the input pattern and the output pattern are different: $\mathbf{y}^p \neq \mathbf{x}^p$.

Auto-associative memory: the stored input pattern and the output pattern are identical: $\mathbf{y}^p = \mathbf{x}^p$. When a pattern $\mathbf{x}$ is presented, the memory retrieves the pattern $\mathbf{x}^p$ that resembles it most closely.

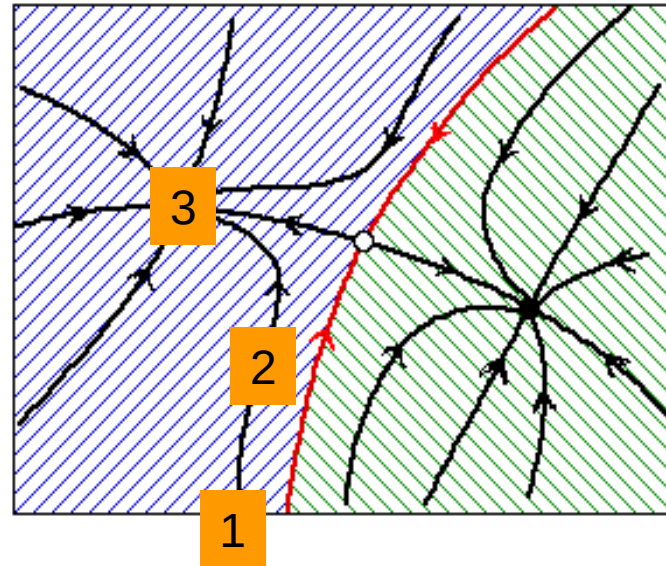
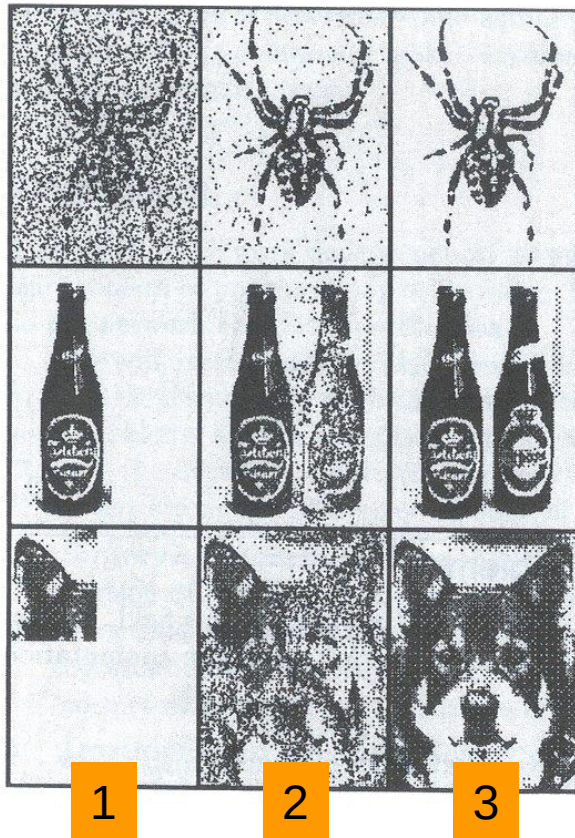
Applied to the previous examples:

- Telephone directory is hetero-associative.
- Completing a web page based on a fragment of text is auto-associative.
- Face recognition has both components: the completion of the face is auto-associative, and recalling the name of the person is hetero-associative.

Auto-associative memories specialise in pattern completion.

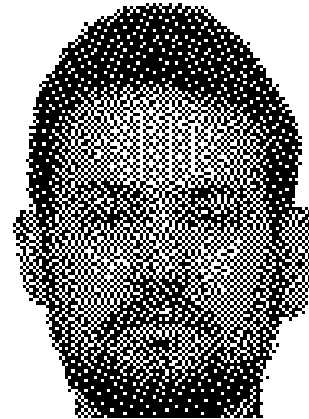
Pattern completion

Stored patterns (3) can be viewed as attractors in state space.



Pattern completion example

Auto-associative neural network that has been trained with two different faces as input patterns [\[demo\]](#).



Properties of associative memories

Shared with the brain:

- Content addressable (unlike standard computer memory)
- Insensitive to small errors in the input patterns (noise resistance)
- Insensitive to removal of network connections and units (graceful degradation)
- Highly parallel processing of information

Different from the brain:

- Usually there is a separate **Learning Phase** (when the associations are stored) and a **Recall Phase** (when the stored information is retrieved)

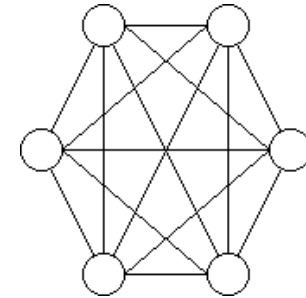
Associative memory models

A number of associative memory models exist in the neural network literature, e.g. Taylor (1956), Steinbuch (1961), Anderson (1968), Gabor (1969), Kohonen (1971), Nakano & Nagumo (1971), Gardner-Medwin (1976).

The two most influential ones are the Hopfield Net and the Associative or Willshaw Net.

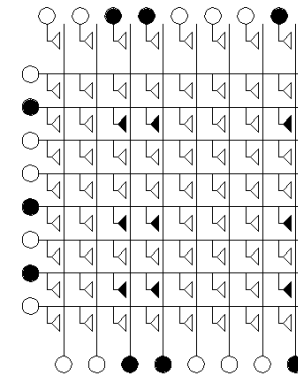
Hopfield Net (1982)

- Auto-associative
- Recurrent
- Origin in physics (statistical mechanics)
- Stability of network states can be determined using an energy function



Associative Net (1969)

- Hetero-associative
- Feed-forward (usually)
- Origin in theoretical biology
- Capacity can be analysed using information theory approach

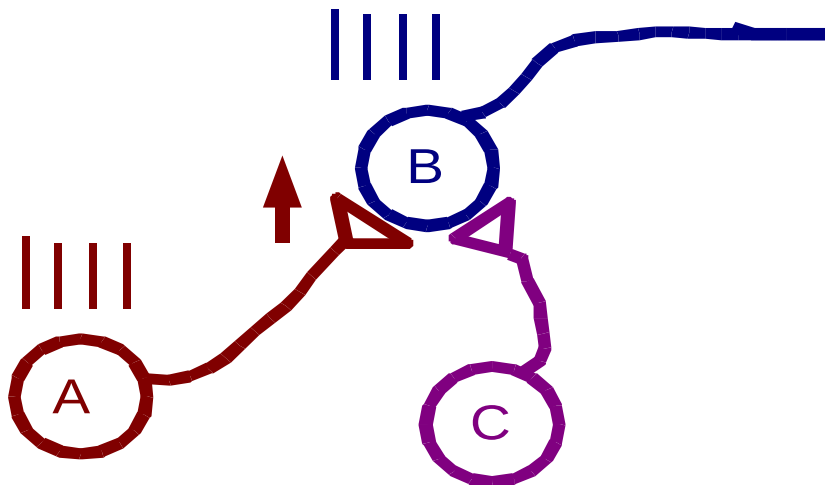
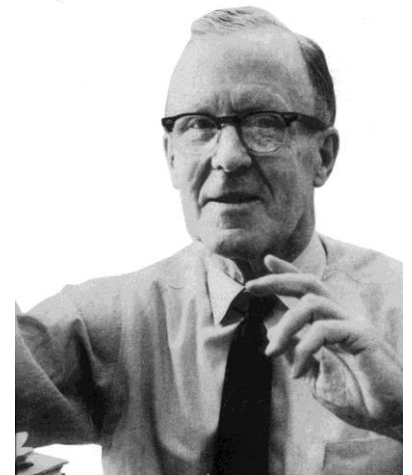


All of these associative memory models are based on **Hebbian Learning**.

Hebbian Learning

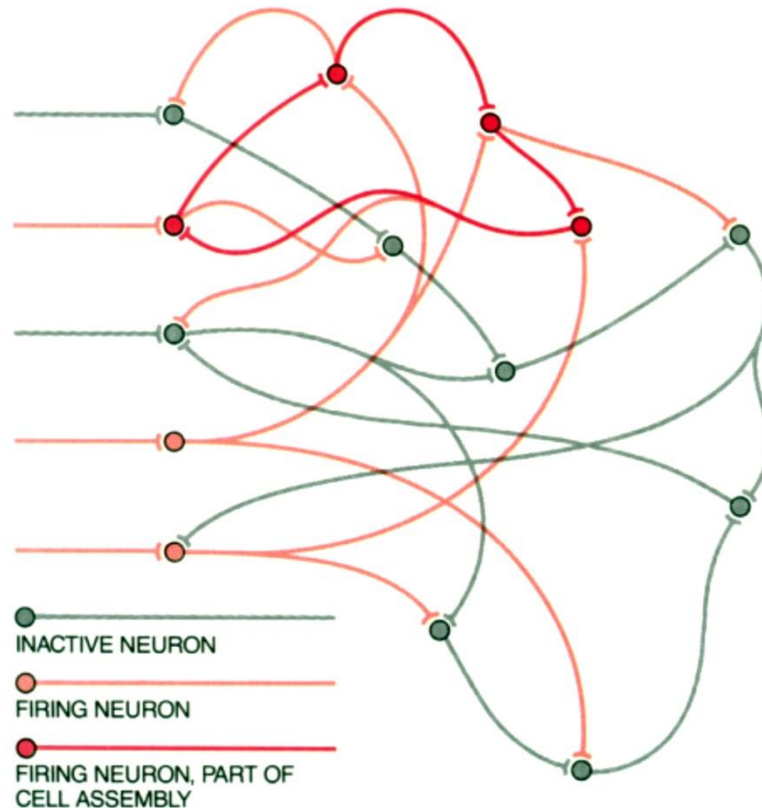
Suggested by the Canadian psychologist Donald Hebb in 1949 as the basis of learning and memory in the brain. Hebb postulated that coincident pre- and postsynaptic activity results in strengthening of the synapses between the pre- and postsynaptic neuron:

“When an axon of cell A is near enough to excite cell B or repeatedly or consistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A’s efficiency, as one of the cells firing B, is increased.”



Hebbian learning and cell assemblies

Hebb (1949): the strengthening of synapses between active neurons leads to the formation of functional cell assemblies (groups of neurons that fire together).

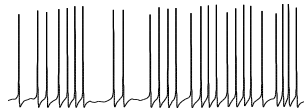


“Neurons that fire together wire together.”

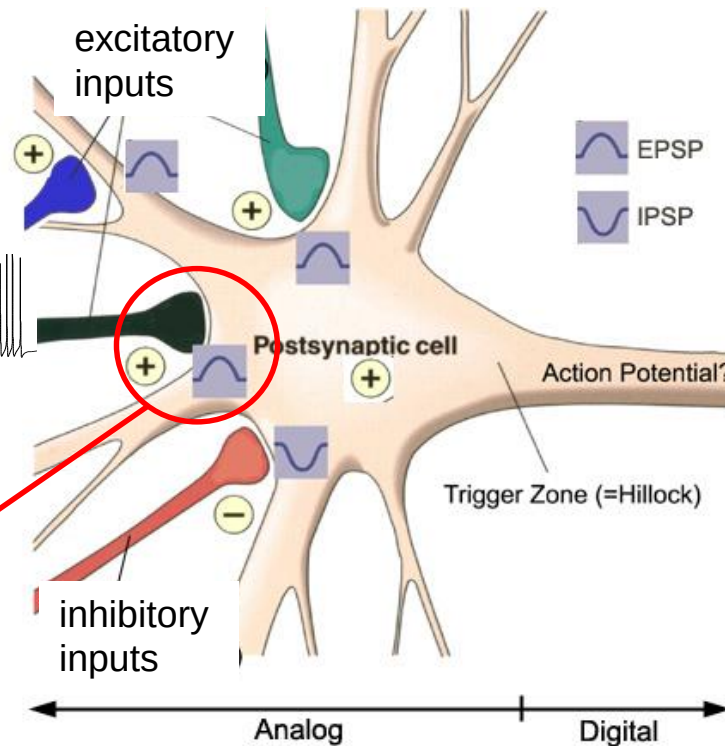
Experimental evidence for Hebb's Rule

Bliss & Lømo (1973): experimental study of synaptic plasticity.

Electrically stimulate
input to the neuron



Measure resulting
postsynaptic potentials
(EPSPs)



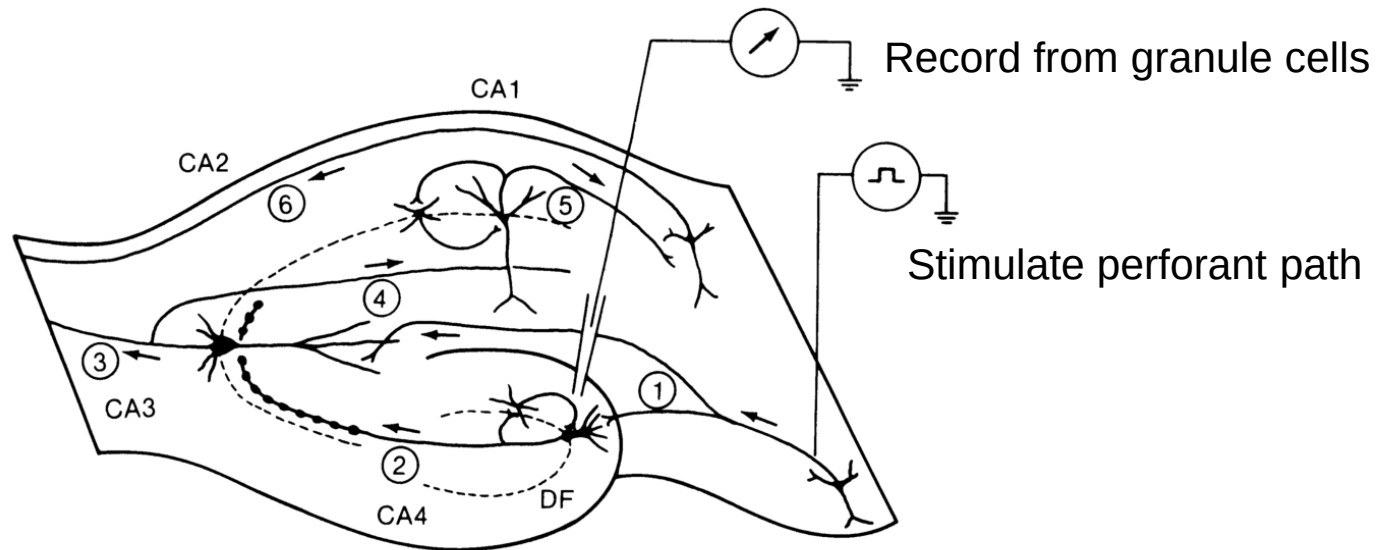
Hebbian learning in the hippocampus

The hippocampus is a brain region that is involved in learning and memory.

Removal of the hippocampus (e.g. as treatment of epilepsy) can result in anterograde amnesia (loss of ability to store new information).

Bliss & Lømo (1973):

- Stimulate perforant path axons with high-frequency (tetanic) shocks
- Record from their target neurons (dentate gyrus granule cells)



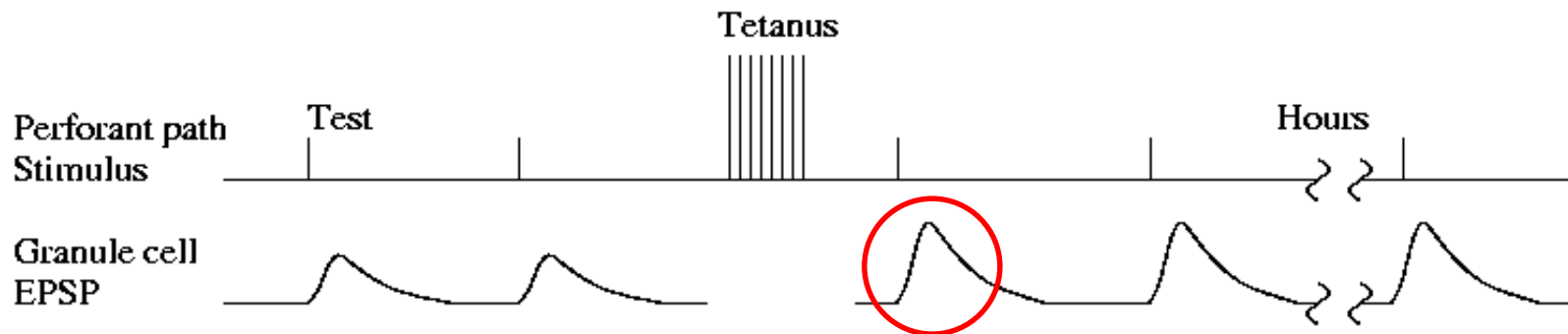
Long-term potentiation (LTP)

After tetanic stimulation: granule cells respond to test stimuli with larger postsynaptic potentials (EPSPs).

The increase in response amplitude can last for hours and even days.

This phenomenon has therefore been called long-term potentiation (LTP).

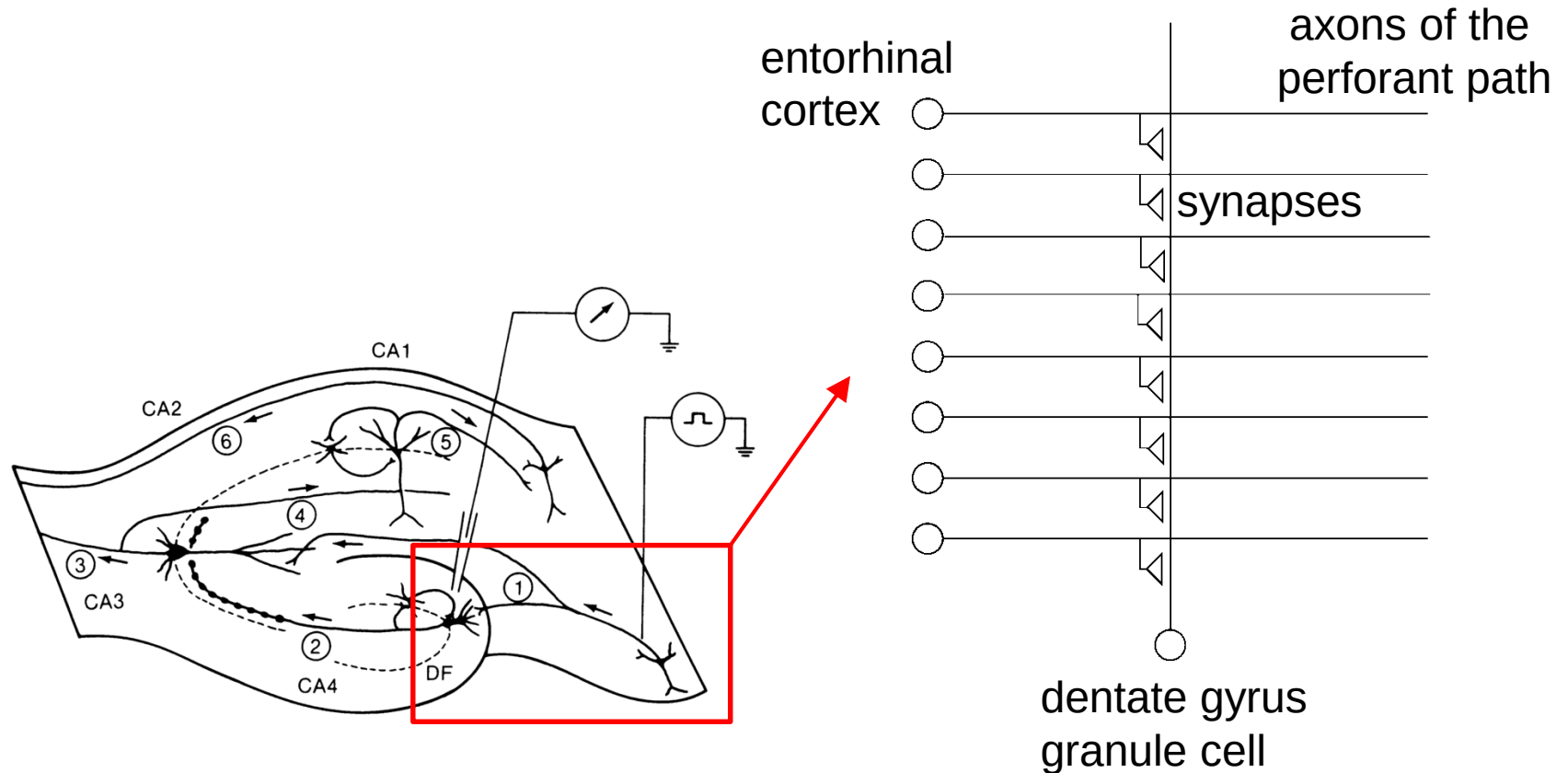
Specificity: LTP is limited to the synapses that have been activated.



What is the relationship to Hebb's rule?

LTP and Hebbian learning

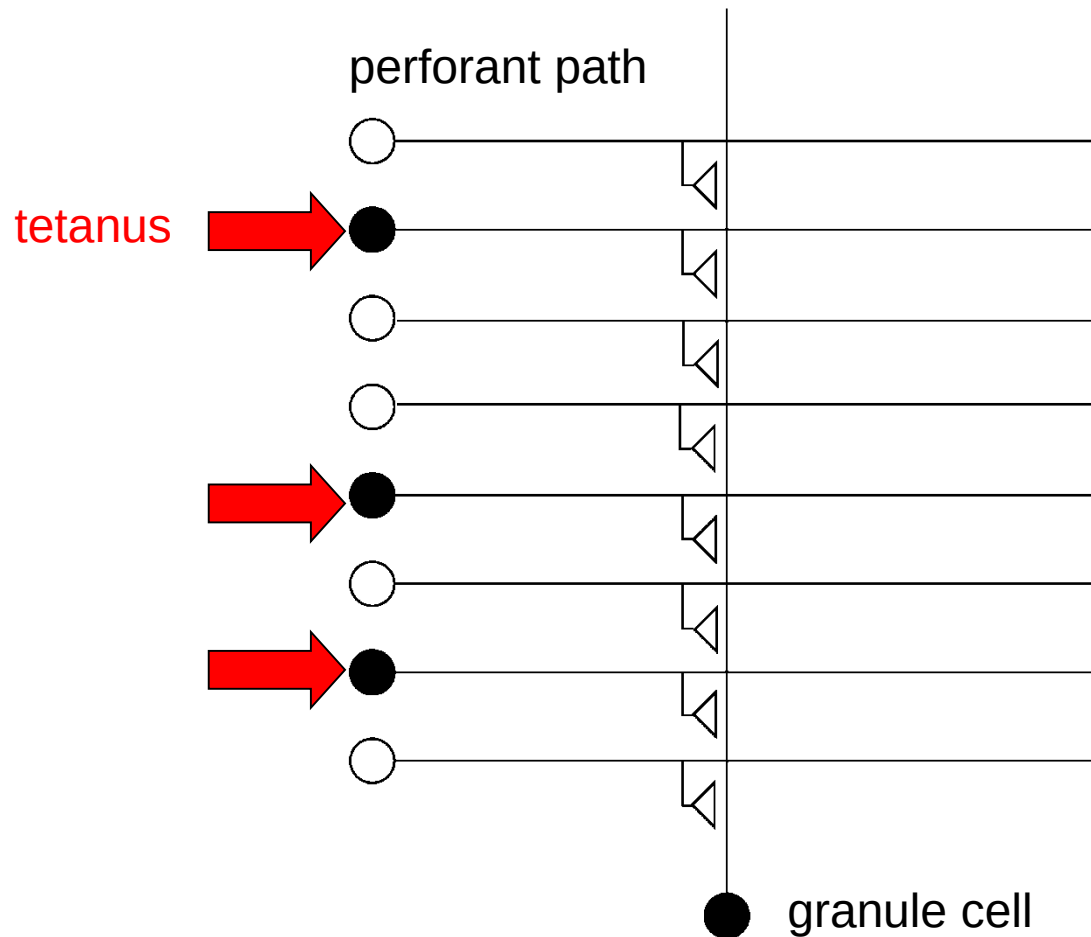
Neural circuitry of the relevant part (input layer) of the hippocampus.



This is a very simple feed-forward neural network.

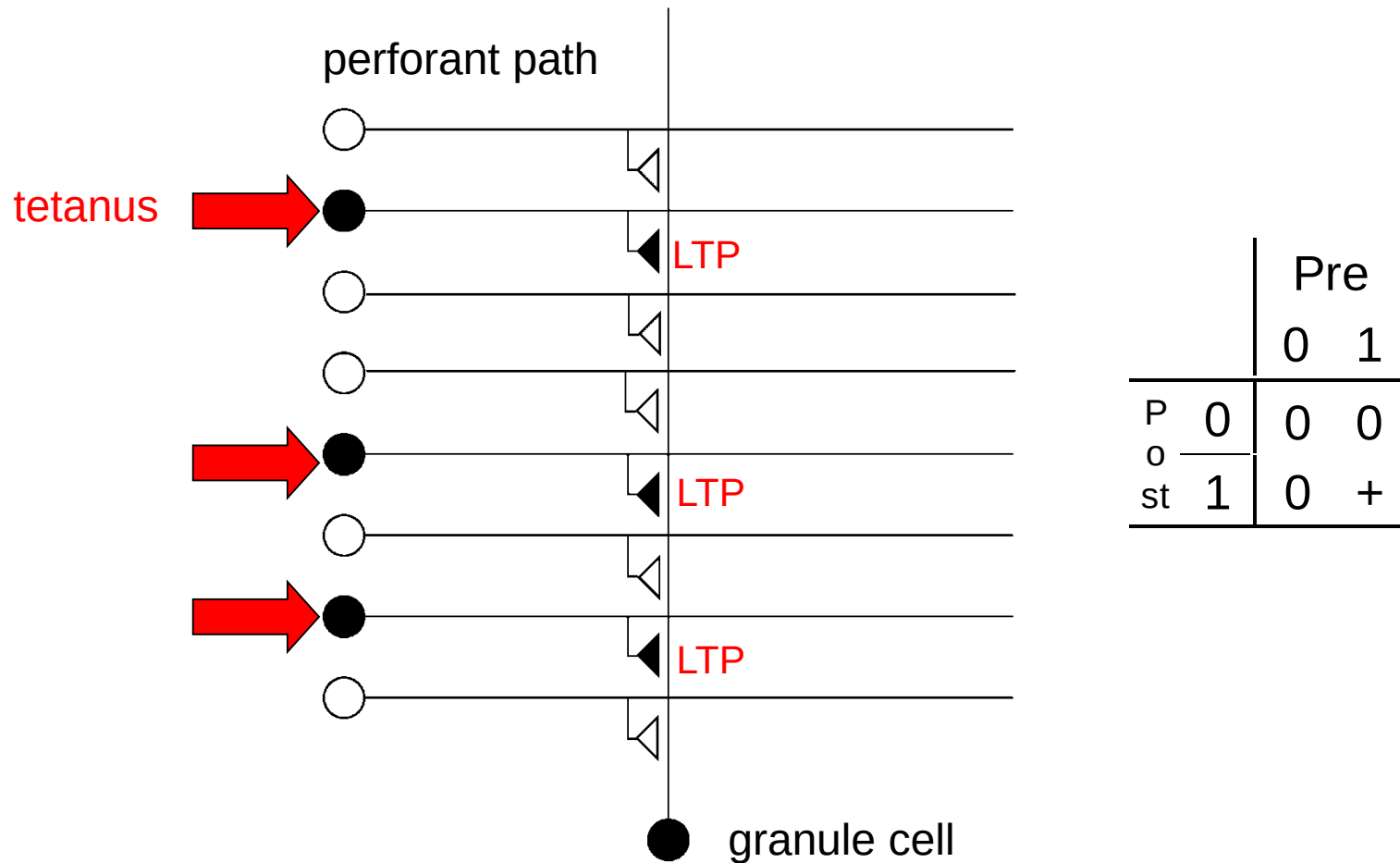
Induction of Hebbian LTP in the hippocampus

Tetanic stimulation of the perforant path results in coincident activation of pre- and postsynaptic neurons (black dots indicate activation).



Hebbian LTP is based on pre- and postsynaptic activity

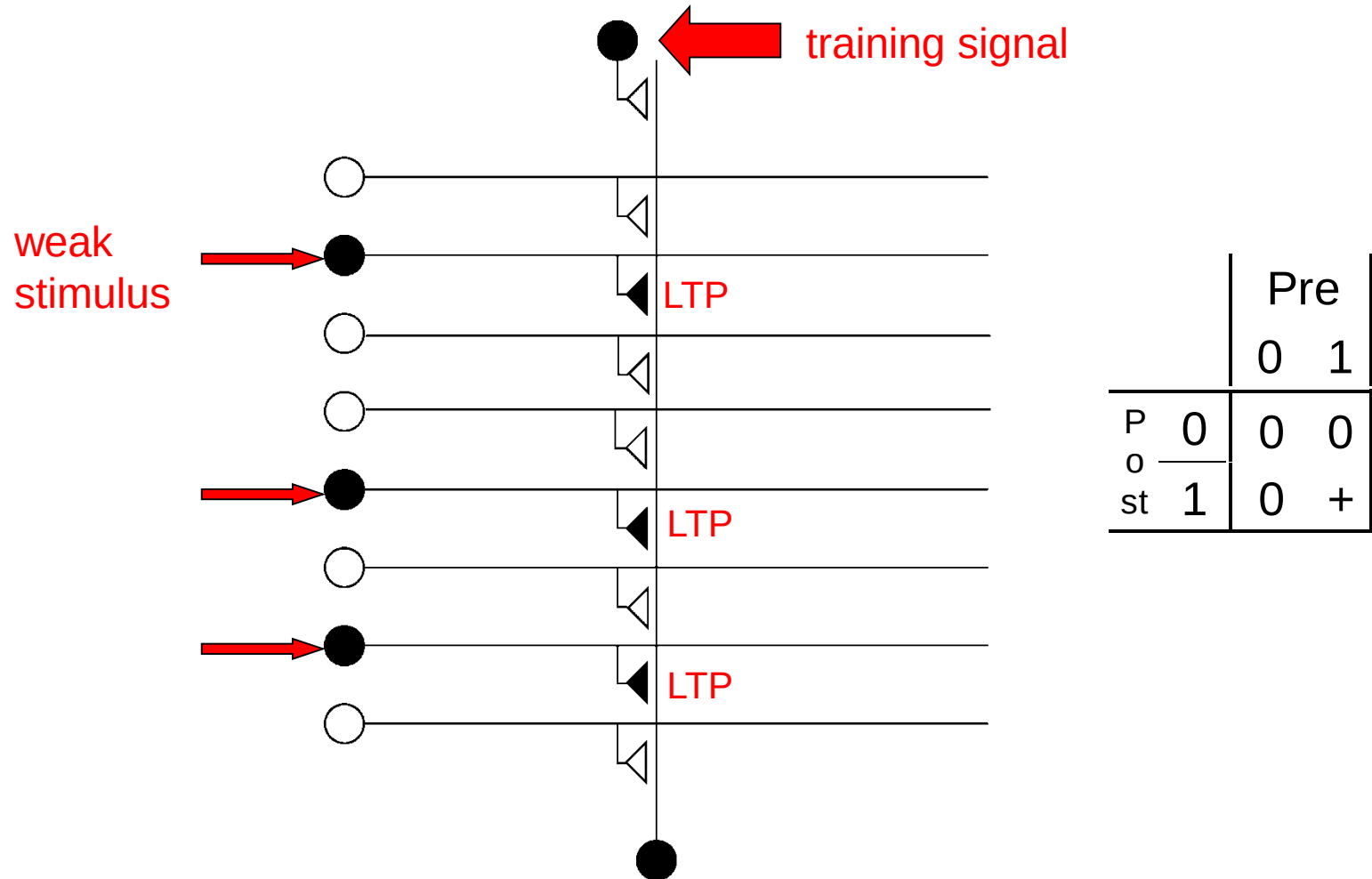
Coincident activation of pre- and postsynaptic neurons induces LTP (Hebb's rule).



But not all types of LTP in the brain require postsynaptic activity!

Training signals

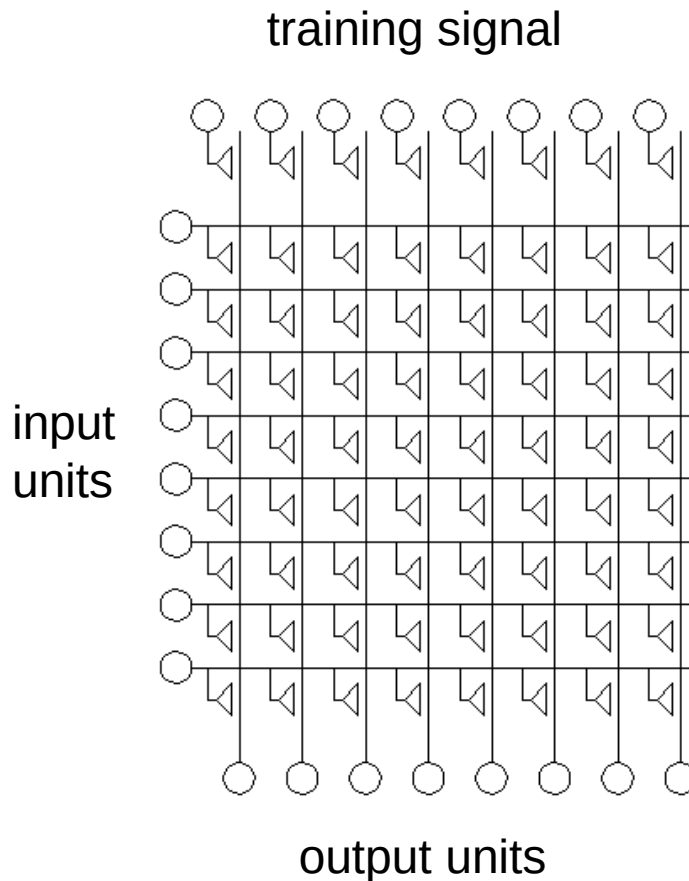
Applying tetanic stimulation is not always necessary to induce LTP. The tetanic stimulus can be replaced by pairing weaker inputs with a training signal.



How can this be used to store and recall patterns in neural nets?

The Associative Net (Willshaw 1969)

- Hetero-associative memory
- Feed-forward connections
- Fully connected
- Binary input and output units
- Binary synapses
- Clipped Hebbian learning: synaptic weights switch from 0 to 1 when pre- and postsynaptic activity coincides

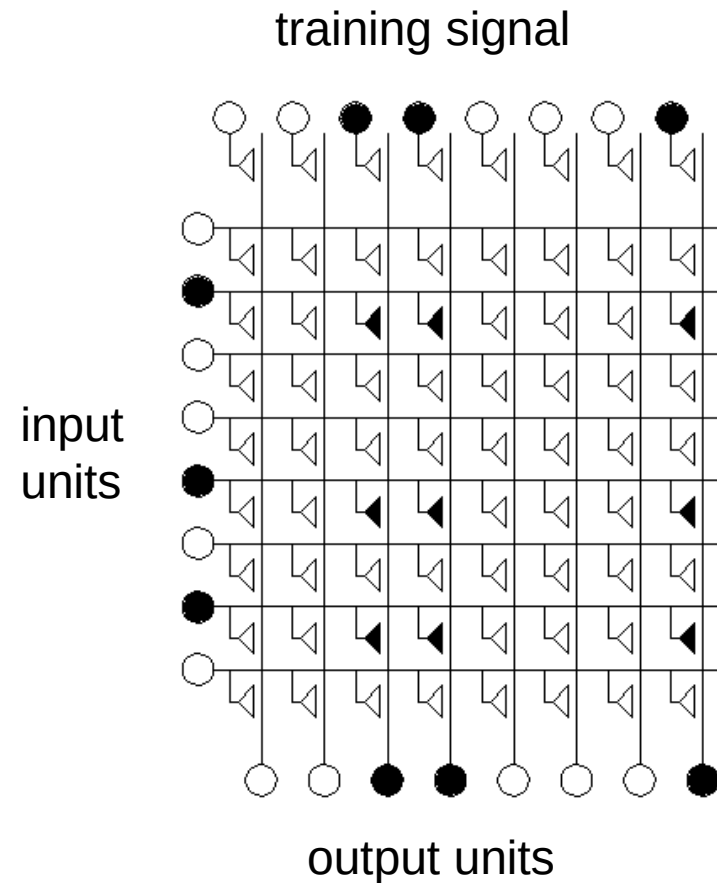


Willshaw et al., Nature 222: 960-962 (1969)

Willshaw, PhD thesis, Edinburgh (1971)

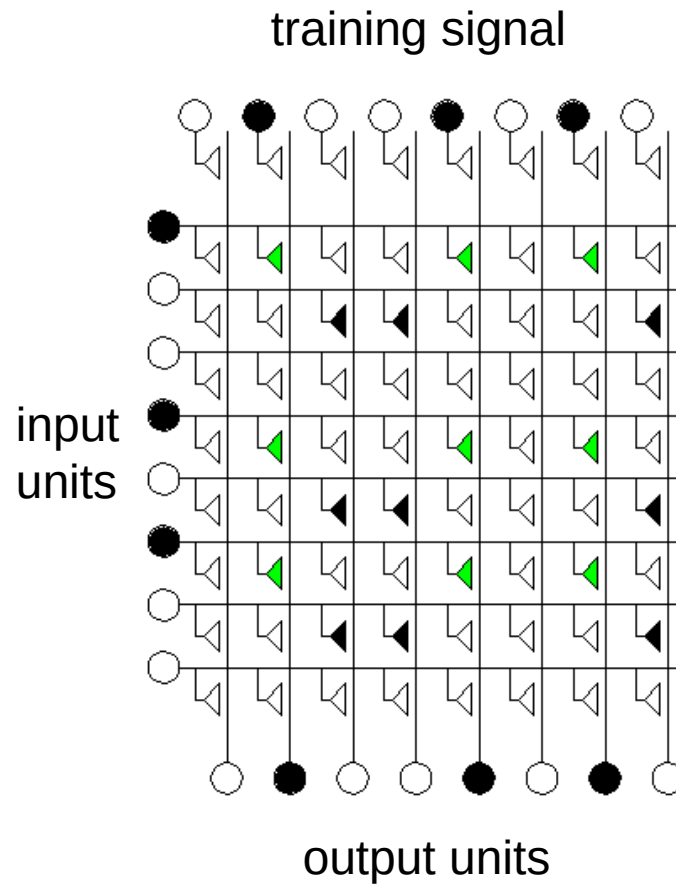
Learning one input-output association

- Initially: all synapses are switched off (set to 0)
- Learning: present an input pattern together with a target output pattern (target can be determined e.g. by training signal)
- Input pattern: binary string of length N_I with M_I active components (set to 1).
- Output pattern: binary string of length N_O with M_O active components.
- Example: $N_I = N_O = 8$, $M_I = M_O = 3$, input pattern (01001010), output pattern (00110001).
- Hebbian Learning: switch on synapses between active input and output units (indicated in black).



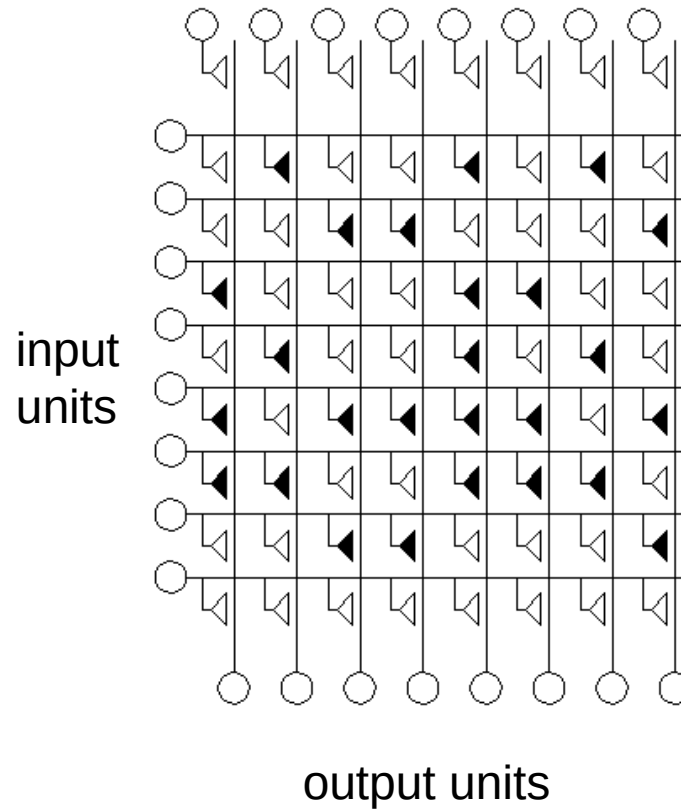
Learning a second association

Storing the association between input (10010100) and output (01001010) switches on the synapses that are shown in green.



After having stored three associations

How many synapses have been switched on?



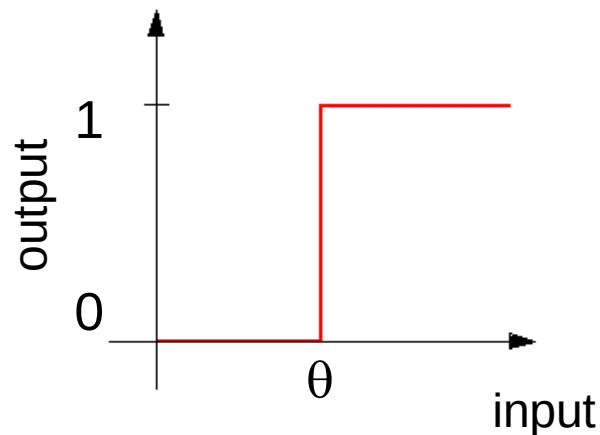
Recalling stored patterns

Present a stored input pattern with the aim of recalling the output pattern that was paired with it during the learning phase.

For each output unit, we need to decide whether or not it should be switched on.

As the associations have been stored by switching on synapses, output units should be on if they receive sufficient input (i.e. if the input sum is larger than or equal to a threshold θ).

The output units are binary threshold units:

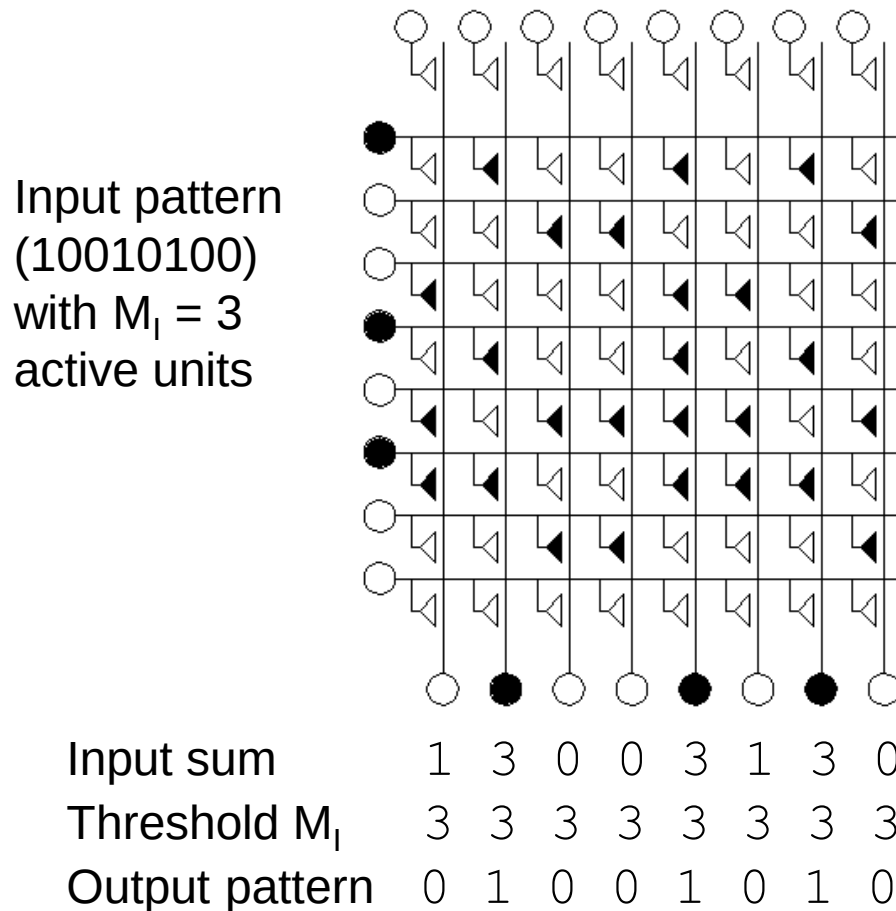


How to calculate the input sum and how to determine the threshold θ ?

Setting the recall threshold

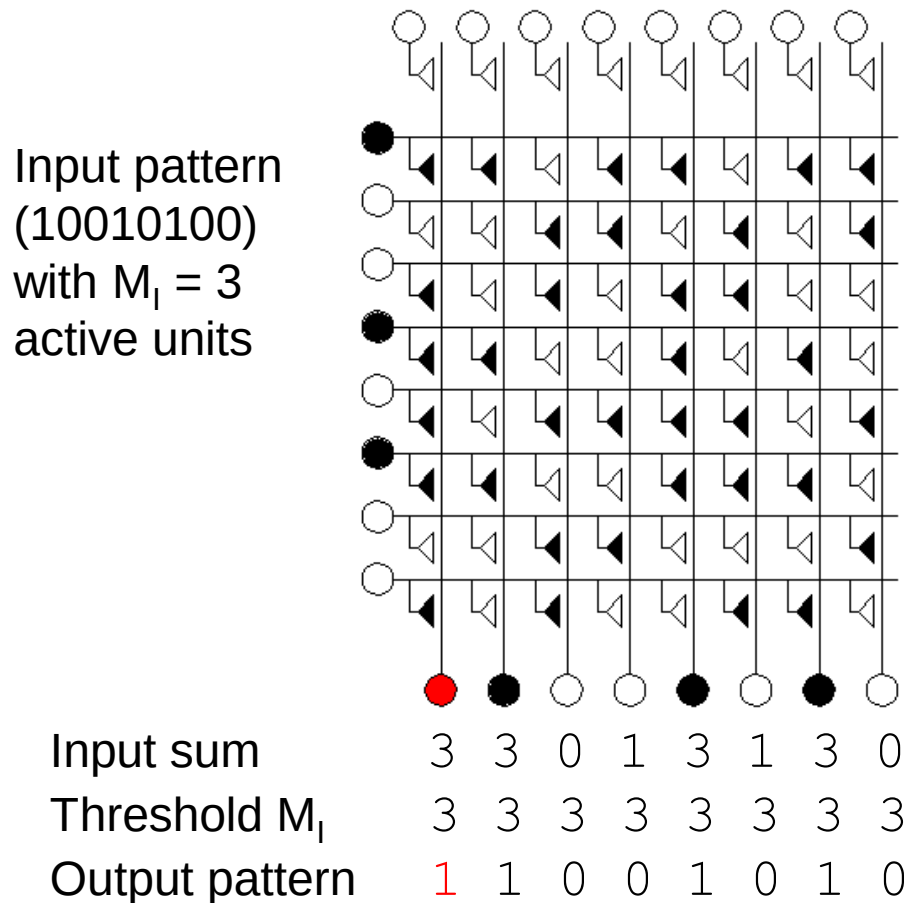
For each output unit, the input sum is given by the number of active input units that are connected to it through synapses that have been switched on (i.e. the inner product of input vector and weight vector).

Threshold is set to the number of active input units per pattern (here $M_i = 3$).



Saturation of the Associative Net

Storing too many associations leads to false positive outputs (indicated in red).



Capacity of the Associative Net

What is the probability that a given synapse has been turned on when the net has stored R associations between input patterns and output patterns?

Input patterns: M_I of the N_I inputs units are on.

Output patterns: M_O of the N_O output units are on.

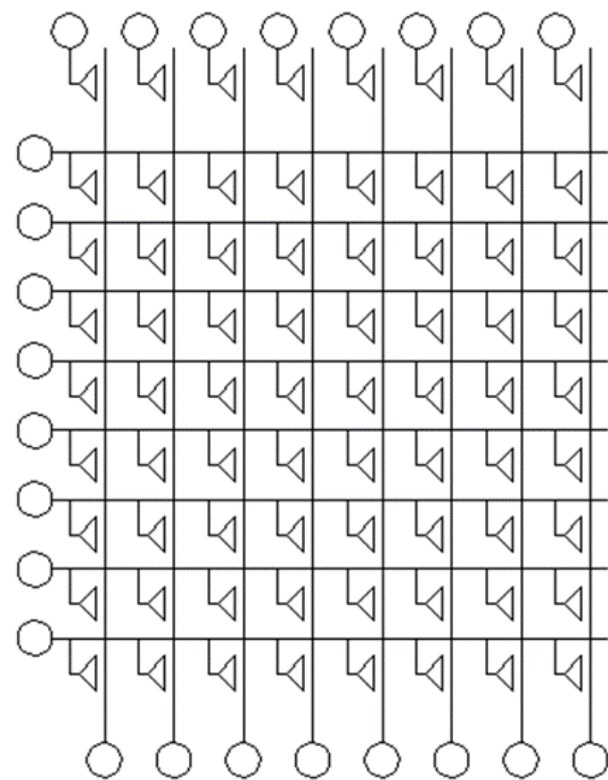
Net contains $N_I N_O$ connections. Storing one association turns on $M_I M_O$ connections.

Probability that a given connection is turned on: $\frac{M_I M_O}{N_I N_O}$

Probability that a given connection is not turned on: $1 - \frac{M_I M_O}{N_I N_O}$

After the storage of R associations, the probability that a given connection is not turned on: $\left(1 - \frac{M_I M_O}{N_I N_O}\right)^R$

Probability P that a connection is turned on: $P = 1 - \left(1 - \frac{M_I M_O}{N_I N_O}\right)^R$



Capacity of the Associative Net (2)

Can calculate how the number of stored associations depends on the probability P that a connection has been turned on and on the size and sparseness of the input and output patterns N_I , N_O , M_I , M_O .

Approximate P by
$$P = 1 - \left(1 - \frac{M_I M_O}{N_I N_O} \right)^R \approx 1 - \exp \left(- \frac{R M_I M_O}{N_I N_O} \right)$$

Rearranging:
$$R \approx - \frac{N_I N_O}{M_I M_O} \ln(1 - P)$$

An error occurs during recall if an output unit that should not be active receives input from all M_I inputs units through synapses that have been turned on.

This occurs with probability P^{M_I}

Number of output units that should not be on: $N_O - M_O$

Average output error: $P^{M_I} (N_O - M_O)$

Capacity of the Associative Net (3)

Good recall if average output error ≤ 1 .

Thus we want $P^{M_I} (N_O - M_O) \leq 1$

Rearranging:
$$P \leq \left(\frac{1}{N_O - M_O} \right)^{1/M_I}$$

Given this criterion we can calculate the capacity of the net (i.e. R = the number of patterns that can be stored before the error becomes too large).

For $N_I = N_O = 1000$, $M_I = M_O = 100$:

$$P \leq 0.93, R \leq 272$$

For $N_I = N_O = 1000$, $M_I = M_O = 10$:

$$P \leq 0.5, R \leq 7000$$

Sparse coding is more efficient!

Increasing the biological realism

The vanilla form of the Associative Net is fully connected.

Real neural networks in the brain are incompletely connected. Estimates for the hippocampus are in the range of 5%.

Need to change the method of threshold setting to deal with incomplete connectivity.

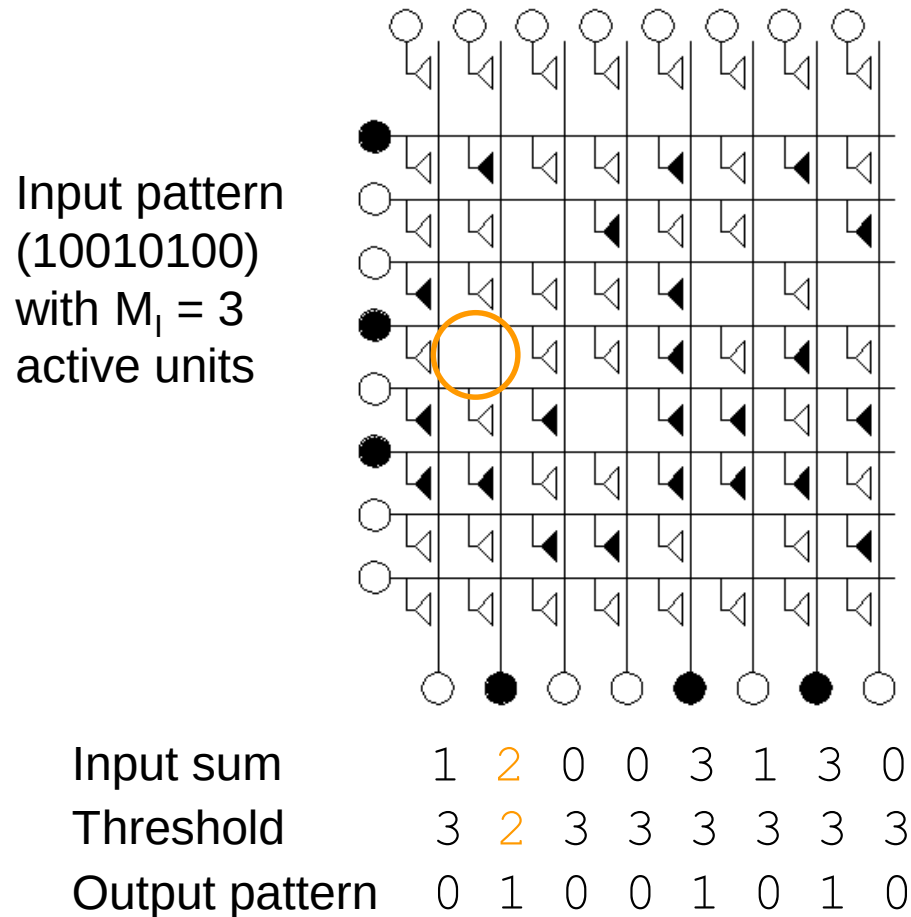
Previously: threshold was set to the number of active input units M_i .

Missing synapses mean that not all output units that should be on will receive M_i active inputs.

Alternative: set threshold for each output unit to the number of active input units that are connected to it through existing synapses (i.e. the maximal input the output unit can actually receive).

Partially connected associative net

Local threshold setting in an associative net with missing synapses: each output unit needs to know the maximum input it can receive.



Noisy inputs

Partial connectivity is easy to solve, but it is not the only complicating factor in associative networks.

Real (and artificial) neural networks operate in the presence of two other types of noise:

- Spuriously active input units
- Missing genuine input units

This makes threshold setting more complicated. Can use different **Winner-take-all (WTA)** approaches:

- Switch on the M_O output units with the highest input sum
- Switch on the M_O output units with the highest ratio of input sum to the number of active input units connected to them through existing synapses

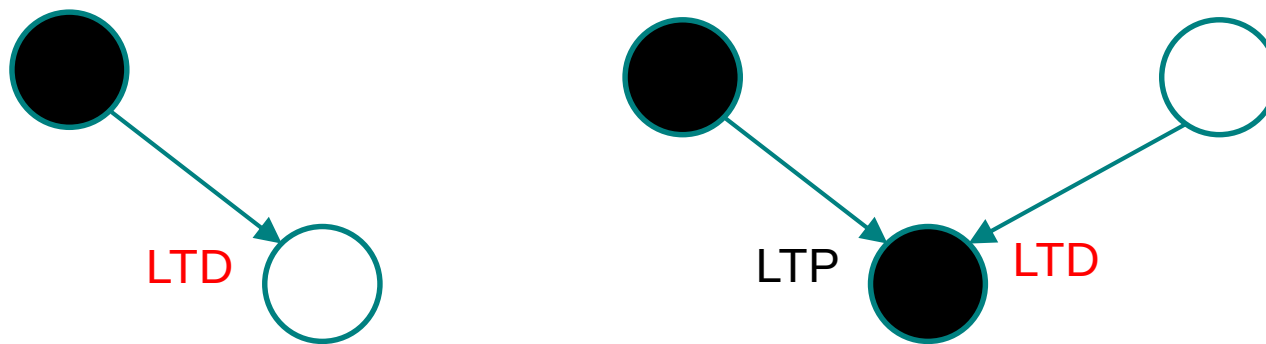
Are these approaches biologically realistic?

Long-term depression

So far we have looked at learning based on Hebbian LTP (coincident pre- and postsynaptic activity leads to increased synaptic weights).

In real synapses, the synaptic weight can also be decreased. Two main types of long-term depression (LTD):

- Homosynaptic LTD: when presynaptic activity fails to elicit postsynaptic activity.
- Heterosynaptic LTD: depression of the unstimulated inputs when LTP-inducing stimuli are applied to another set of inputs.



LTD increases storage capacity

Depression is not “forgetting”: it helps separate patterns and increases the memory capacity of the associative network.

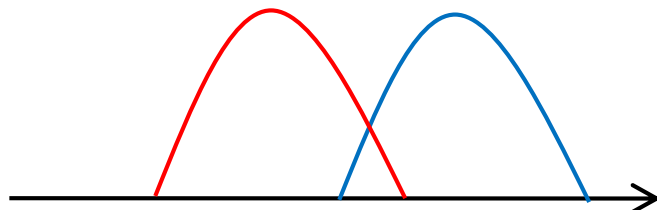
Can use homosynaptic LTD, heterosynaptic LTD or a combination of both:

		Pre	
		0	1
P o st	0	0	-
	1	0	+

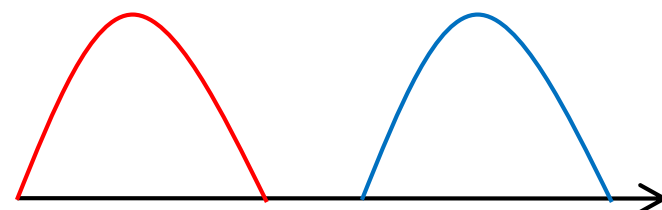
		Pre	
		0	1
P o st	0	0	0
	1	-	+

		Pre	
		0	1
P o st	0	α	β
	1	γ	δ

Analysis shows that optimal pattern storage is achieved with a combination of increases and decreases in synaptic efficacy.

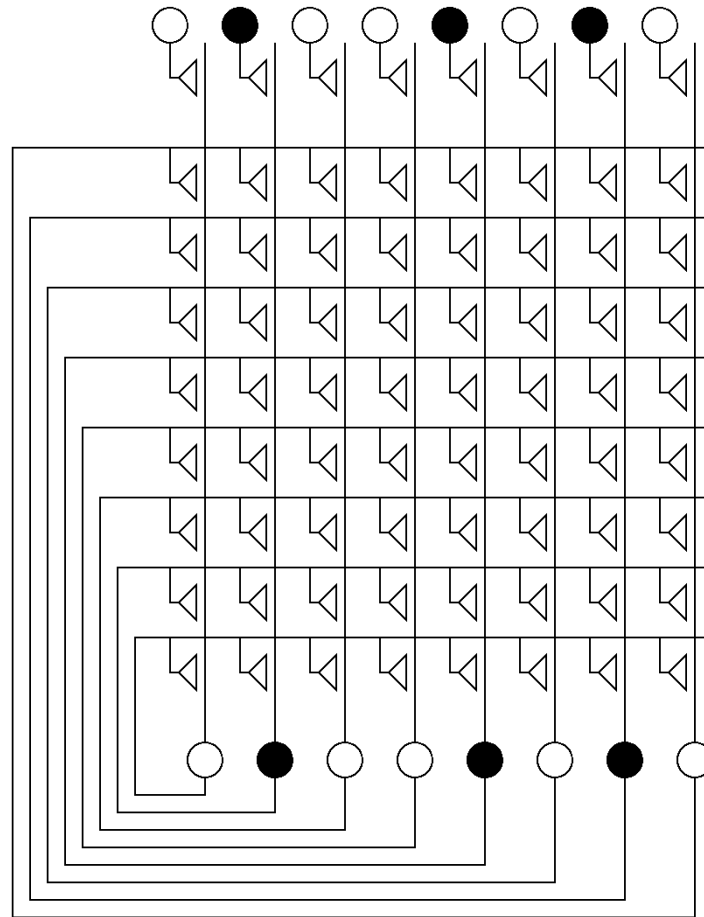


Input sum without LTD



Input sum with LTD

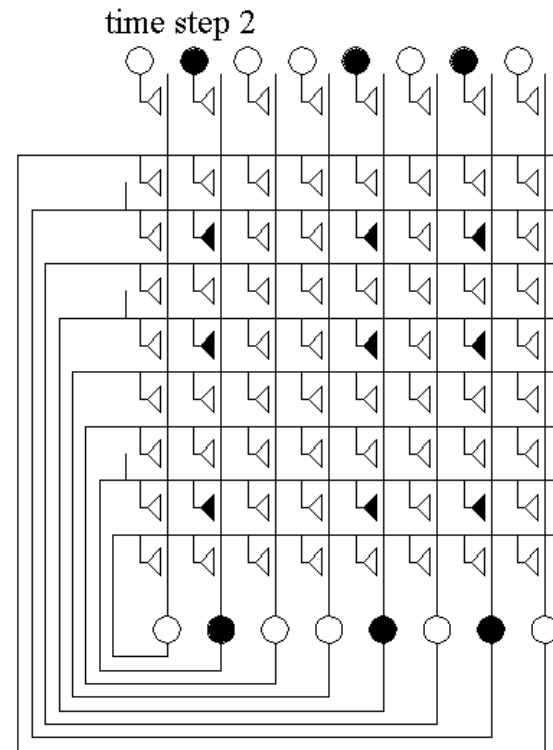
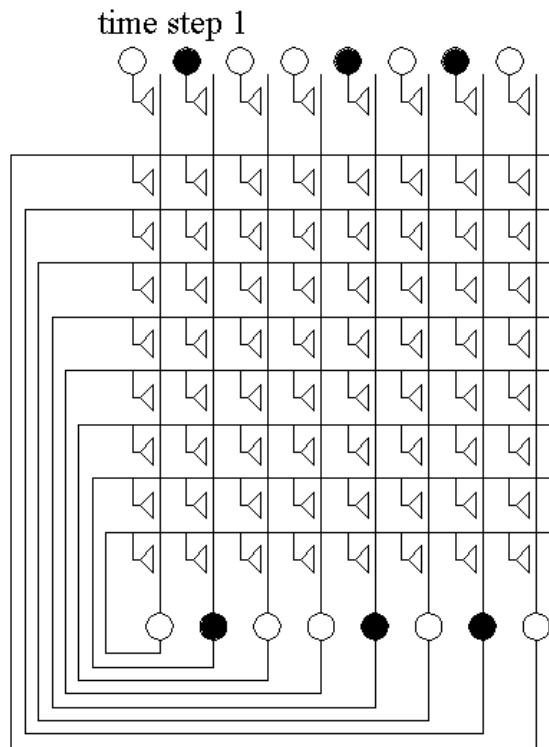
A recurrent associative network



What can such a recurrent associative network learn?

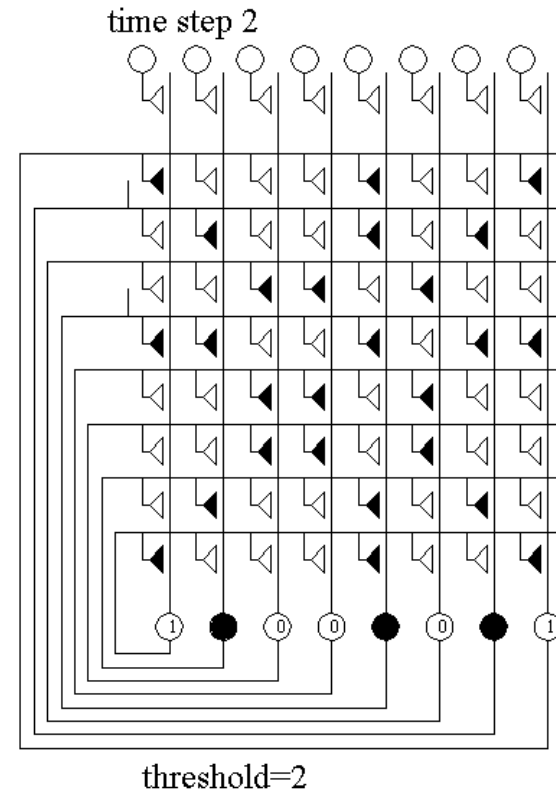
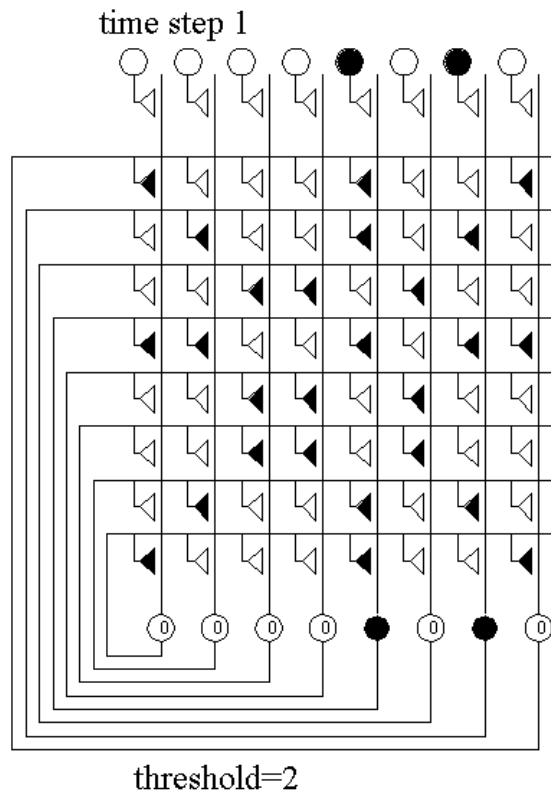
Learning with a steady input

Auto-associative memory: learning one pattern.



Recalling auto-associative memories

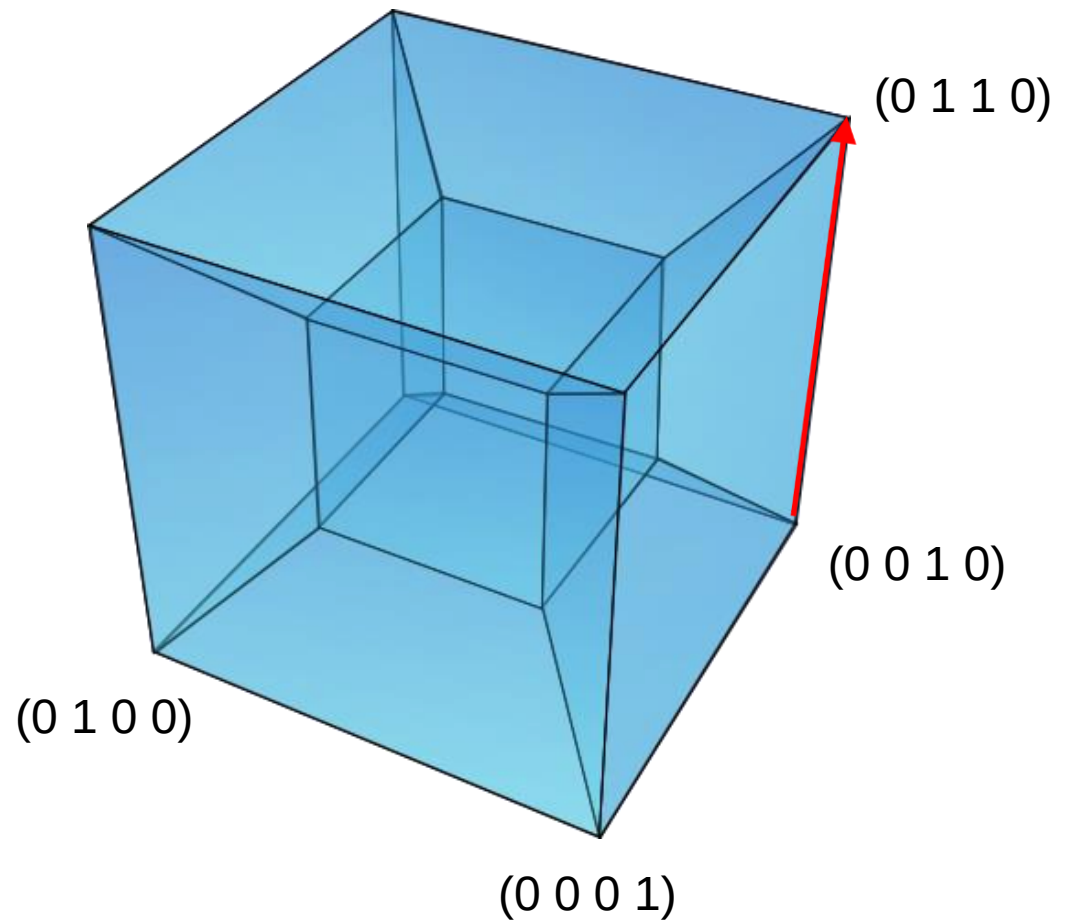
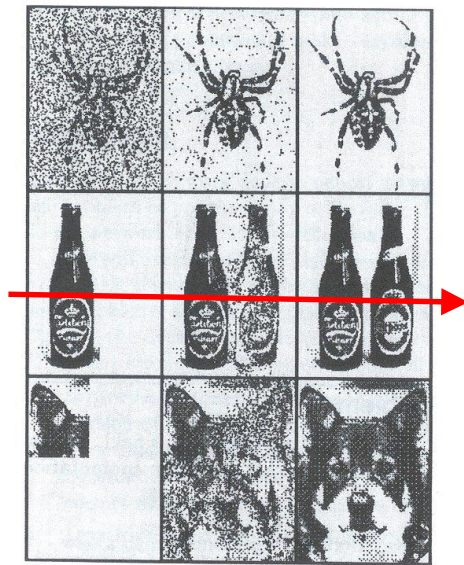
Recalling a pattern from an incomplete cue.



State space

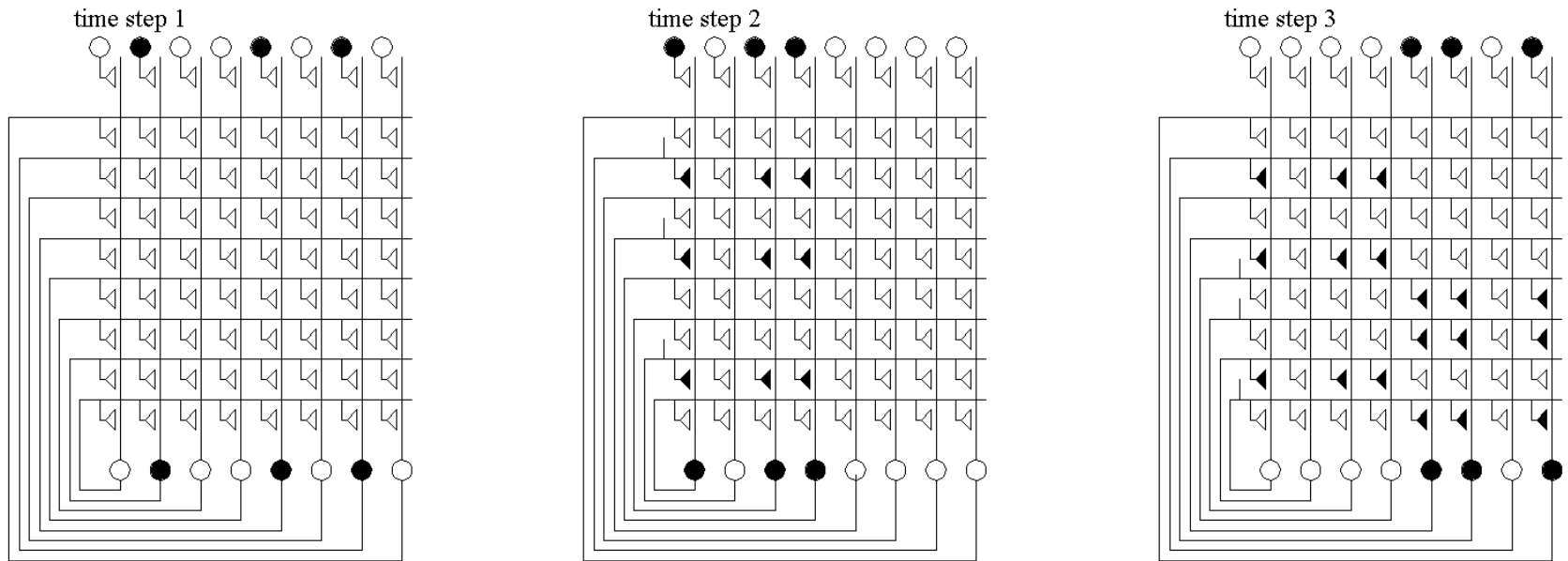
The recurrent associative net stores N_O -dimensional patterns.

The state space is therefore an N_O -dimensional hypercube (here for $N_O = 4$).



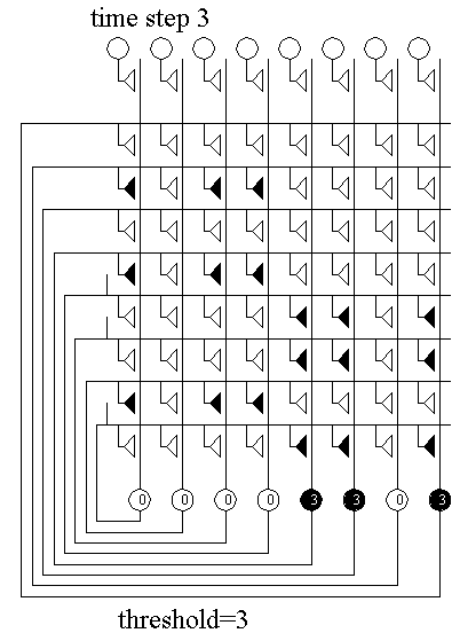
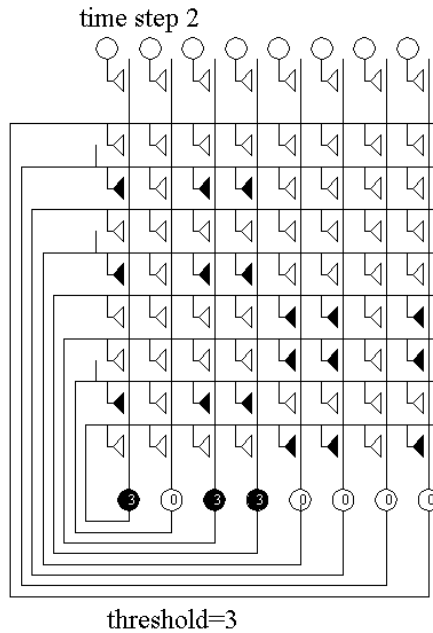
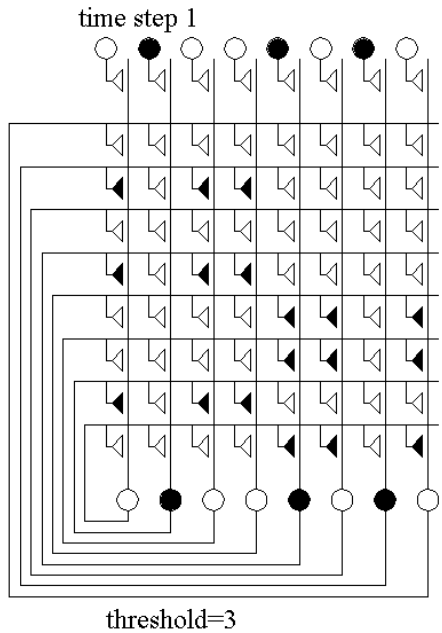
Learning with changing input patterns

Sequence of hetero-associative memories.



Recalling a sequence of patterns

Sequence can be recalled by presenting the first pattern.



Bibliography

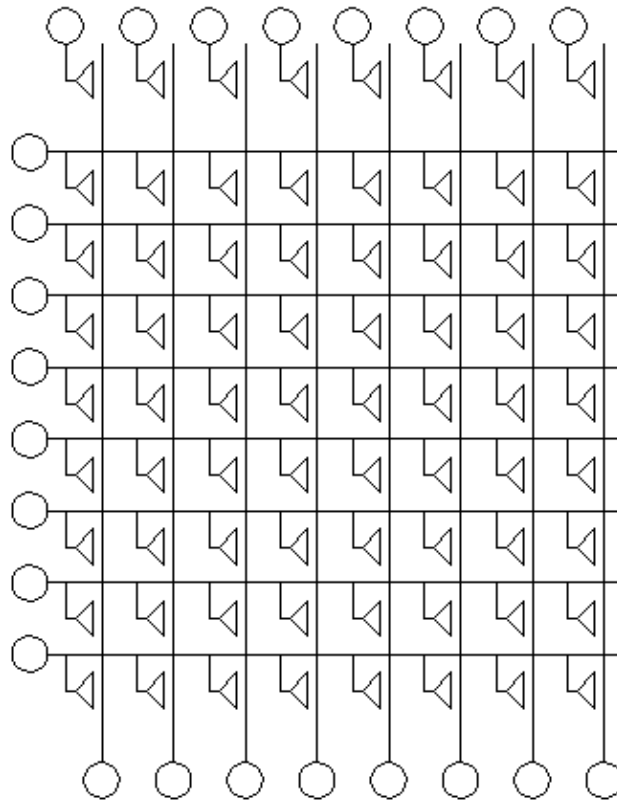
- Willshaw et al., Nature 222: 960-962 (1969)
- Hertz, Krogh & Palmer. Introduction to the theory of neural computation. Addison-Wesley, 1991.
- Gurney. An introduction to neural networks. CRC press, 1997.
- Churchland & Sejnowski. The computational brain. MIT press, 1992.
- Rumelhart & McClelland. Parallel Distributed Processing. MIT press, 1986.

Summary?

Exercise 1

Store two input-output associations in the associative net. Present one of the stored input patterns and check if the correct output pattern is recalled.

Suggested associations: $\text{input}_1 = (01001001)$, $\text{output}_1 = (11000001)$;
 $\text{input}_2 = (10101000)$, $\text{output}_2 = (01011000)$. Test with input_1 .



Exercise 2

Store two other associations. Present the stored input pattern from the previous exercise again and check if the correct output pattern is recalled.

Suggested associations: $\text{input}_1 = (01001001)$, $\text{output}_1 = (11000001)$;
 $\text{input}_2 = (10101000)$, $\text{output}_2 = (01011000)$; $\text{input}_3 = (00010011)$, $\text{output}_3 = (00101010)$;
 $\text{input}_4 = (11000100)$, $\text{output}_4 = (01001100)$. Test with input_1 .

